

# **COMPUTER SCIENCE PROJECT**

**NAME: FALAK MINOCHA**

**ROLL NO: 26BA110769**

**BRANCH: B.TECH CSE AI ML**



along



along

# ULTIMATE QUIZ ARENA

THE ULTIMATE GAME SYSTEM

**Name: FALAK MINOCHA**

**BATCH: 2026-30**

| <b>S. No.</b> | <b>Description</b>                   | <b>Page No.</b> |
|---------------|--------------------------------------|-----------------|
| <b>1.</b>     | <b>ACKNOWLEDGEMENT</b>               | <b>4</b>        |
| <b>2.</b>     | <b>CERTIFICATE</b>                   | <b>5</b>        |
| <b>3.</b>     | <b>INTRODUCTION</b>                  | <b>6</b>        |
| <b>4.</b>     | <b>COMMERCIAL SCOPE</b>              | <b>7</b>        |
| <b>5.</b>     | <b>PYTHON FRONT END</b>              | <b>9</b>        |
| <b>6.</b>     | <b>USER DEFINED FUNCTIONS (UDFs)</b> | <b>11</b>       |
| <b>7.</b>     | <b>ALGORITHM</b>                     | <b>13</b>       |
| <b>8.</b>     | <b>FLOWCHART</b>                     | <b>16</b>       |
| <b>9.</b>     | <b>CODE</b>                          | <b>17</b>       |
| <b>10.</b>    | <b>OUTPUTS</b>                       | <b>156</b>      |
| <b>11.</b>    | <b>BIBLIOGRAPHY</b>                  | <b>202</b>      |

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me in completing my Python project, **"Ultimate Quiz Arena."**

I am thankful to my faculty members for providing me with the opportunity to work on this project and for guiding me throughout its development. Their suggestions helped me understand how Python programming concepts can be combined to create an interactive and useful application.

I would also like to thank my classmates and friends who helped me test different parts of the program and provided useful suggestions regarding the questions, menu structure and overall user experience.

This project helped me improve my understanding of important Python concepts such as variables, lists, dictionaries, conditional statements, loops, functions, user input and the random module.

Finally, I am grateful to my institution for providing me with the learning environment and resources required to complete this project successfully.

FALAK MINOCHA

26BA110769

# INTRODUCTION

## 3.1 Project Overview

Ultimate Quiz Arena is an interactive, console-based quiz and mini-game application developed using Python.

The main purpose of the project is to make quiz-based learning more interesting by combining normal question answering with game-like features. Instead of providing only a simple question-and-answer system, the project includes different quiz modes, difficulty levels, power-ups, experience points, coins, achievements, statistics and mini-games.

The project is designed using beginner-level Python concepts so that its working can be easily understood and explained.

The program uses the random module to select questions and generate certain game activities. All information is maintained during the current program session using Python data structures. No SQL database, file handling or external database is used.

## 3.2 Subjects

The quiz contains questions from four main subjects:

- Python
- Mathematics
- Science
- General Knowledge

Each subject contains questions divided into:

- Easy
- Medium

- Hard

This allows the player to experience questions with different levels of difficulty.

### **3.3 Quiz Modes**

The project provides multiple ways to play:

1. Classic Quiz
2. Rapid Fire
3. Survival
4. Daily Challenge
5. Boss Battle
6. Tournament
7. Category Mix

Each mode provides a slightly different gameplay experience.

### **3.4 Game Hub**

Apart from quizzes, the project contains a Game Hub with:

- Quick Math
- Number Hunt
- Rock Paper Scissors

These activities provide additional ways for the player to earn points and coins.

### **3.5 Power-Ups**

The quiz also provides power-ups to make the gameplay more interactive.

The available power-ups are:

- 50/50

- Hint
- Phone
- Skip
- Double
- Shield

The player receives a limited number of power-ups and can also interact with the Arena Shop.

### **3.6 Player Progress**

The program maintains several player statistics during the current session, including:

- Score
- XP
- Level
- Correct answers
- Wrong answers
- Questions attempted
- Current streak
- Best streak
- Number of quizzes
- Games played
- Coins
- Boss wins
- Tournament progress

- Achievements

This makes the application more game-oriented than a basic quiz program.

### **3.7 Objective of the Project**

The main objectives are:

- To develop an interactive Python application.
- To apply basic Python programming concepts practically.
- To create an engaging quiz environment.
- To provide multiple subjects and difficulty levels.
- To demonstrate the use of functions.
- To use lists and dictionaries for structured data.
- To use randomization for question selection.
- To maintain player statistics during a session.
- To provide a simple game-based learning experience.



# COMMERCIAL SCOPE

The current Ultimate Quiz Arena is a console-based academic project. However, its basic concept can be expanded into a larger educational application.

## 4.1 Educational Applications

The project can be adapted for:

- School quizzes
- College practice tests
- Competitive examination preparation
- Programming practice
- General knowledge competitions
- Classroom activities
- Self-learning applications

Different question banks could be created for different subjects and courses.

## 4.2 Online Quiz Platform

The current application can be extended into a web-based platform where students could access quizzes through a browser.

Possible additions include:

- User accounts
- Online question banks

- Online leaderboards
- Teacher dashboards
- Result history
- Timed examinations
- Online competitions

### **4.3 Mobile Application**

The same concept could be developed as a mobile application.

The existing game structure could be converted into a graphical interface containing:

- Buttons
- Question cards
- Score displays
- Progress bars
- Achievement badges
- Game animations

### **4.4 Institutional Use**

Educational institutions could use an expanded version for conducting:

- Department quizzes
- Inter-class competitions
- Programming contests
- General knowledge competitions

- Practice examinations

## **4.5 Future Scope**

Future versions could include:

- A larger question database
- User login
- Persistent scores
- Online multiplayer
- Database integration
- Graphical user interface
- AI-based question generation
- Detailed performance analytics
- Cloud-based leaderboards

These features are future possibilities and are not part of the current basic version.

# PYTHON FRONT END

## 5.1 Introduction

The front end of Ultimate Quiz Arena is a **console-based user interface** developed completely in Python.

The user interacts with the program through the terminal by entering numerical choices and answering questions.

## 5.2 Technologies Used

| Technology             | Purpose                                       |
|------------------------|-----------------------------------------------|
| Python                 | Main programming language                     |
| random module          | Random question and game selection            |
| Lists                  | Storing questions, options and records        |
| Dictionaries           | Storing player and question information       |
| Functions              | Dividing the program into manageable sections |
| Conditional statements | Handling choices and decisions                |
| Loops                  | Repeating menus and quiz questions            |
| input()                | Taking user input                             |
| print()                | Displaying information                        |

## **5.3 Main Menu**

The application provides the following main menu:

=== ULTIMATE QUIZ ARENA ===

1. Quiz Modes
2. Game Hub
3. Arena Shop
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit

Each option calls a separate user-defined function.

## **5.4 Quiz Modes Menu**

The quiz menu contains:

1. Classic Quiz
2. Rapid Fire
3. Survival
4. Daily Challenge
5. Boss Battle
6. Tournament

7. Category Mix

8. Return

## **5.5 Game Hub**

The Game Hub contains:

1. Quick Math

2. Number Hunt

3. Rock Paper Scissors

4. Return

## **5.6 Arena Shop**

The Arena Shop allows the player to interact with the available power-ups and the Risk and Reward option.

The shop uses coins maintained in the player's session data.

# UDFs :

## (USER DEFINED FUNCTIONS)

- 1 . title()    Displays headings and section titles
- 2 . num()    Takes and validates numerical input
- 3 . wait()    Pauses the program before continuing
- 4 . level()    Selects the quiz difficulty level
- 5 . choose\_subject()    Selects the quiz subject
- 6 . ask()    Displays a question and processes the answer
- 7 . quiz()    Runs the main quiz system
- 8 . modes()    Displays and handles quiz modes
- 9 . games()    Handles Game Hub activities
- 10 . shop()    Handles Arena Shop and power-ups
- 11 . stats()    Displays player statistics
- 12 . achieve()    Displays earned achievements
- 13 . board()    Displays the session leaderboard
- 14 . help()    Displays the Arena Guide
- 15 . main()    Controls the main program and menu

# ALGORITHM

Step 1: Start the program.

Step 2: Initialize the player information, question bank, scores and available power-ups.

Step 3: Display the Ultimate Quiz Arena title.

Step 4: Display the main menu.

Step 5: Take the player's choice.

Step 6: If the player selects Quiz Modes, open the quiz mode menu.

Step 7: Select the desired quiz mode.

Step 8: Select the subject and difficulty where required.

Step 9: Select questions from the question bank.

Step 10: Display each question with its options.

Step 11: Take the player's answer.

Step 12: Check the answer.

Step 13: Update score, XP, correct answers, wrong answers and streak.

Step 14: Apply power-ups if selected.

Step 15: Continue until the quiz is completed.



Step 16: If Game Hub is selected, display the available mini-games.

Step 17: If Arena Shop is selected, display power-ups and available shop options.

Step 18: If Player Stats is selected, display the player's current statistics.

Step 19: If Achievements is selected, display earned achievements.

Step 20: If Leaderboard is selected, display the current session scores.

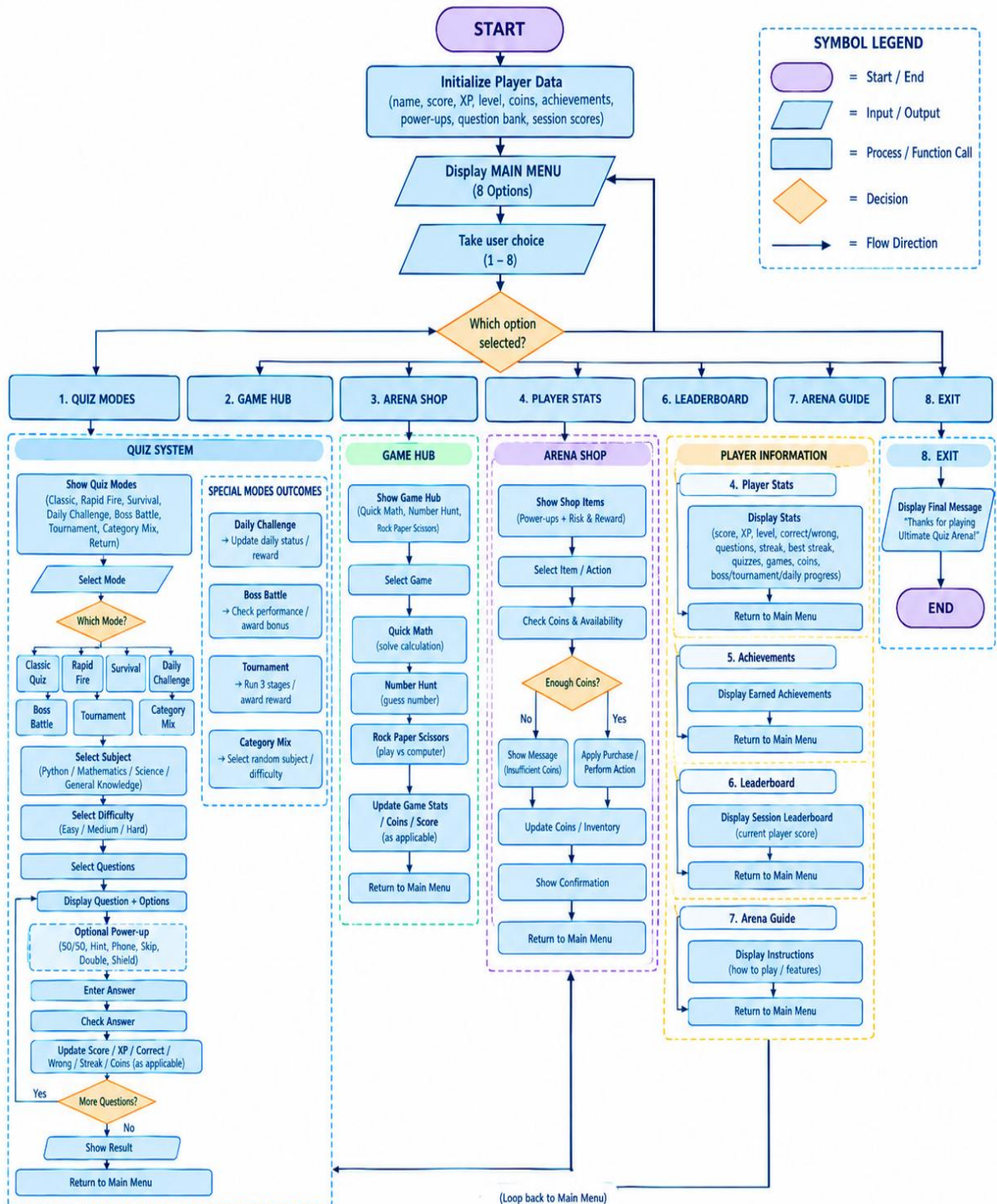
Step 21: If Arena Guide is selected, display the instructions.

Step 22: Return to the main menu.

Step 23: If Exit is selected, display the exit message.

Step 24: End the program.

# FLOWCHART



# CODE

```
##### ULTIMATE QUIZ ARENA #####
import random
import time

player = {
    "name": "Player", "score": 0, "xp": 0, "level": 1,
    "correct": 0, "wrong": 0, "questions": 0, "streak": 0,
    "best_streak": 0, "quizzes": 0, "boss_wins": 0, "games": 0,
    "lifelines": 0, "phone_help": 0, "double_used": False,
    "achievements": [], "perfect_rounds": 0, "coins": 0,
    "tournament_wins": 0, "daily_done": False, "rounds_won": 0,
    "best_speed": None, "total_time": 0, "questions_answered": 0,
    "powerups": {"50/50": 2, "Hint": 2, "Phone": 2, "Skip": 1, "Double": 1, "Shield": 1}
}

leaderboard = []

questions = {
    "Python": [
        "Easy": [
            ("What is the output of print(3 * 'Hi')?", ["HiHiHi", "3Hi", "Hi3", "Error"], "A", "A string can be repeated with the * operator."),
            ("Which data type stores unique values?", ["List", "Tuple", "Set", "String"], "C", "A set stores unique elements."),
            ("What does len([4, 8, 12, 16]) return?", ["3", "4", "5", "16"], "B", "There are four elements in the list."),
            ("Which operator checks equality?", ["=", "==", "!=", ">"], "B", "== compares two values."),
            ("Which keyword defines a function?", ["func", "define", "def", "function"], "C", "Python uses def to define functions."),
            ("What is the first index of a Python list?", ["0", "1", "-1", "It depends"], "A", "Python uses zero-based indexing."),
            ("Which function displays output?", ["input()", "show()", "print()", "display()"], "C", "print() displays information on the screen."),
            ("Which symbol starts a Python comment?", ["//", "#", "*", "-"], "B", "Python comments begin with #."),
            ("What type is the value True?", ["int", "str", "bool", "float"], "C", "True and False are Boolean values."),
            ("Which method adds one item to the end of a list?", ["add()", "push()", "append()", "insertEnd()"], "C", "append() adds an item at the end.")
        ],
        1,
        "Medium": [
            ("x=[10,20]; y=x; y.append(30); print(x)", ["[10,20]", "[30]", "[10,20,30]", "Error"], "C", "x and y refer to the same list object."),
            ("What does [x*2 for x in [1,2,3]] produce?", ["[1,2,3]", "[2,4,6]", "[1,4,9]", "Error"], "B", "The comprehension doubles every element."),
            ("Which creates a shallow copy of list a?", ["b=a", "b=a[:]"], "b=a", "a.copy(deep=True)", "B", "Slicing creates a shallow list copy."),
            ("What does range(2, 8, 2) produce?", ["2,4,6", "2,4,6,8", "1,3,5,7", "2,3,4,5,6,7"], "A", "The stop value 8 is excluded."),
            ("What is the result of 17 // 5?", ["2", "3", "3.4", "4"], "B", "// performs floor division."),
            ("Which statement safely handles an exception?", ["if/else", "try/except", "for/while", "def/return"], "B", "try/except handles runtime exceptions."),
            ("What does dictionary.get('x', 0) return if x is absent?", ["None only", "0", "Error", "x"], "B", "get() can return a default value."),
            ("What is len({'a':1, 'b':2, 'c':3})?", ["2", "3", "4", "6"], "B", "A dictionary has three key-value pairs."),
            ("What does enumerate({'a':b}) provide?", ["Only values", "Only indexes", "Indexes and values", "Keys and values"], "C", "enumerate() yields index-value pairs."),
            ("Which is immutable?", ["List", "Set", "Dictionary", "Tuple"], "D", "Tuples cannot be changed after creation.")
        ],
        1,
        "Hard": [
            ("What is printed by: a=[1]; b=a.copy(); b.append(2); print(a,b)?", ["([1] [1,2])", "[1,2] [1,2]", "[1] [2]", "Error"], "A", "copy() creates a separate outer list."),
            ("d={'a':10}; print(d.get('b',0)+d['a']) gives:", ["0", "10", "20", "Error"], "B", "The missing key returns 0, so 0+10=10."),
            ("What does a function return when it reaches return without a value?", ["0", "False", "None", "Error"], "C", "A bare return produces None."),
            ("What is the output of print([i for i in range(6) if i % 2 == 0])?", ["[1,3,5]", "[0,2,4]", "[2,4,6]", "[[]]", "B", "The condition keeps even numbers from 0 through 5."),
            ("Which statement about a shallow copy is correct?", ["Nested objects are always copied", "Only the outer container is copied", "Nothing is copied", "It converts a list to a tuple"], "B", "Only the outer container is copied."),
            ("What is the value of bool([])?", ["True", "False", "None", "Error"], "B", "Empty containers are falsy."),
            ("What does 'args' collect in a function?", ["Keyword arguments", "Positional arguments", "Only strings", "Return values"], "B", "'args' collects extra positional arguments."),
            ("Which data structure is best suited for key-based lookup?", ["Dictionary", "Tuple", "String", "Float"], "A", "Dictionaries map keys to values."),
            ("If a=[1,2,3], what is a[-1]?", ["1", "2", "3", "Error"], "C", "-1 refers to the last element."),
            ("What does sorted([3,1,2], reverse=True) return?", ["[1,2,3]", "[3,2,1]", "(3,2,1)", "Error"], "B", "sorted() returns a new list in descending order.")
        ],
        1,
        "Mathematics": {
            "Easy": [
                ("What is 12 x 8?", ["86", "96", "108", "112"], "B", "12x8=96."),
                ("What is 144/9?", ["10", "11", "12", "14"], "C", "12x12=144."),
                ("What is 25% of 200?", ["25", "40", "50", "75"], "C", "25% is one fourth, so the answer is 50."),
                ("What is 7*7?", ["14", "21", "49", "56"], "C", "7x7=49."),
                ("What is 15 + 27?", ["32", "40", "42", "44"], "C", "15+27=42."),
                ("What is 3/4 as a percentage?", ["25%", "50%", "75%", "80%"], "C", "3/4 = 0.75 = 75%)."),
                ("What is the perimeter of a square of side 6 cm?", ["12 cm", "18 cm", "24 cm", "36 cm"], "C", "Perimeter=4xside=24 cm."),
                ("What is 2^3?", ["10", "16", "25", "32"], "D", "2x2x2=2x32."),
                ("What is the next prime after 7?", ["8", "9", "10", "11"], "D", "11 is the next prime number."),
                ("What is 0.5 x 20?", ["5", "10", "15", "20"], "B", "Half of 20 is 10.")
            ],
            1,
            "Medium": [
                ("If 3x=27, what is x?", ["6", "8", "9", "12"], "C", "27=3x9."),
                ("What is the derivative of x^3?", ["x", "2x", "x^1", "2"], "B", "By the power rule, d(x^3)/dx=2x^2."),
                ("What is the average of 10,20,30?", ["15", "20", "25", "30"], "B", "60=3x20."),
                ("What is 2^3+3^2?", ["13", "15", "17", "19"], "C", "8+9=17."),
                ("Solve: 2x+5=17.", ["5", "6", "7", "8"], "B", "2x=12, so x=6."),
                ("What is the slope of y=3x+2?", ["2", "3", "5", "-3"], "B", "The coefficient of x is the slope."),
                ("What is the area of a triangle with base 10 and height 6?", ["16", "30", "60", "120"], "B", "Area=1/2x10x6=30."),
                ("What is 15% of 80?", ["8", "10", "12", "15"], "C", "0.15x80=12.)."),
                ("If a=2/3 and a=1/b, b is:", ["12", "15", "20", "30"], "B", "The scale factor is 5, so b=15.)."),
                ("What is the median of 3,7,9,12,15?", ["7", "8", "9", "12"], "C", "The middle value is 9.")
            ],
            1,
            "Hard": [
                ("What is 2x dx?", ["x^2+C", "2x^2+C", "x^2C", "2xC"], "A", "The integral of 2x is x^2+C."),
                ("If f(x)=x^3, what is f(2)?", ["8", "10", "12", "14"], "B", "4x4=16.)."),
                ("Determinant of [[2,1],[3,4]] is:", ["5", "6", "8", "11"], "A", "2x4-1x3=5.)."),
                ("What is log_10(1000)?", ["2", "3", "10", "100"], "B", "10^3=1000.)."),
                ("Solve x^2-5x+6=0.", ["1,6", "2,3", "-2,-3", "3,4"], "B", "(x-2)(x-3)=0.)."),
                ("What is the derivative of sin(x)?", ["cos(x)", "-cos(x)", "sin(x)", "-sin(x)"], "A", "The derivative of sin(x) is cos(x)."),
                ("If vectors are perpendicular, their dot product is:", ["1", "-1", "0", "Their magnitudes"], "C", "Perpendicular vectors have dot product zero.)."),
                ("What is the probability of getting two heads in two fair coin tosses?", ["1/2", "1/3", "1/4", "3/4"], "C", "Only HH works out of four equally likely outcomes.)."),
                ("The eigenvalues of an identity matrix are:", ["0 only", "1 only", "-1 only", "Any integer"], "B", "Every eigenvalue of I is 1.)."),
                ("What is lim(x-0) sin(x)/x?", ["0", "1", "x", "Does not exist"], "B", "The standard trigonometric limit equals 1.")
            ],
            1,
        }
    }
}
```

```
##### ULTIMATE QUIZ ARENA #####
import random
import time

player = {
    "name": "Player", "score": 0, "xp": 0, "level": 1,
    "correct": 0, "wrong": 0, "questions": 0, "streak": 0,
    "best_streak": 0, "quizzes": 0, "boss_wins": 0, "games": 0,
    "lifelines": 0, "phone_help": 0, "double_used": False,
    "achievements": [], "perfect_rounds": 0, "coins": 0,
    "tournament_wins": 0, "daily_done": False, "rounds_won": 0,
    "best_speed": None, "total_time": 0, "questions_answered": 0,
    "powerups": {"50/50": 2, "Hint": 2, "Phone": 2, "Skip": 1, "Double": 1, "Shield": 1}
}

leaderboard = []

questions = {
    "Python": [
        "Easy": [
            ("What is the output of print(3 * 'Hi')?", ["HiHiHi", "3Hi", "Hi3", "Error"], "A", "A string can be repeated with the * operator."),
            ("Which data type stores unique values?", ["List", "Tuple", "Set", "String"], "C", "A set stores unique elements."),
            ("What does len([4, 8, 12, 16]) return?", ["3", "4", "5", "16"], "B", "There are four elements in the list."),
            ("Which operator checks equality?", ["=", "==", "!=", ">"], "B", "== compares two values."),
            ("Which keyword defines a function?", ["func", "define", "def", "function"], "C", "Python uses def to define functions."),
            ("What is the first index of a Python list?", ["0", "1", "-1", "It depends"], "A", "Python uses zero-based indexing."),
            ("Which function displays output?", ["input()", "show()", "print()", "display()"], "C", "print() displays information on the screen."),
            ("Which symbol starts a Python comment?", ["//", "#", "*", "-"], "B", "Python comments begin with #."),
            ("What type is the value True?", ["int", "str", "bool", "float"], "C", "True and False are Boolean values."),
            ("Which method adds one item to the end of a list?", ["add()", "push()", "append()", "insertEnd()"], "C", "append() adds an item at the end.")
        ],
        1,
        "Medium": [
            ("x=[10,20]; y=x; y.append(30); print(x)", ["[10,20]", "[30]", "[10,20,30]", "Error"], "C", "x and y refer to the same list object."),
            ("What does [x*2 for x in [1,2,3]] produce?", ["[1,2,3]", "[2,4,6]", "[1,4,9]", "Error"], "B", "The comprehension doubles every element."),
            ("Which creates a shallow copy of list a?", ["b=a", "b=a[:]"], "b=a", "a.copy(deep=True)", "B", "Slicing creates a shallow list copy."),
            ("What does range(2, 8, 2) produce?", ["2,4,6", "2,4,6,8", "1,3,5,7", "2,3,4,5,6,7"], "A", "The stop value 8 is excluded."),
            ("What is the result of 17 // 5?", ["2", "3", "3.4", "4"], "B", "// performs floor division."),
            ("Which statement safely handles an exception?", ["if/else", "try/except", "for/while", "def/return"], "B", "try/except handles runtime exceptions."),
            ("What does dictionary.get('x', 0) return if x is absent?", ["None only", "0", "Error", "x"], "B", "get() can return a default value."),
            ("What is len({'a':1, 'b':2, 'c':3})?", ["2", "3", "4", "6"], "B", "A dictionary has three key-value pairs."),
            ("What does enumerate({'a':b}) provide?", ["Only values", "Only indexes", "Indexes and values", "Keys and values"], "C", "enumerate() yields index-value pairs."),
            ("Which is immutable?", ["List", "Set", "Dictionary", "Tuple"], "D", "Tuples cannot be changed after creation.")
        ],
        1,
        "Hard": [
            ("What is printed by: a=[1]; b=a.copy(); b.append(2); print(a,b)?", ["([1] [1,2])", "[1,2] [1,2]", "[1] [2]", "Error"], "A", "copy() creates a separate outer list."),
            ("d={'a':10}; print(d.get('b',0)+d['a']) gives:", ["0", "10", "20", "Error"], "B", "The missing key returns 0, so 0+10=10."),
            ("What does a function return when it reaches return without a value?", ["0", "False", "None", "Error"], "C", "A bare return produces None."),
            ("What is the output of print([i for i in range(6) if i % 2 == 0])?", ["[1,3,5]", "[0,2,4]", "[2,4,6]", "[[]]", "B", "The condition keeps even numbers from 0 through 5."),
            ("Which statement about a shallow copy is correct?", ["Nested objects are always copied", "Only the outer container is copied", "Nothing is copied", "It converts a list to a tuple"], "B", "Only the outer container is copied."),
            ("What is the value of bool([])?", ["True", "False", "None", "Error"], "B", "Empty containers are falsy."),
            ("What does 'args' collect in a function?", ["Keyword arguments", "Positional arguments", "Only strings", "Return values"], "B", "'args' collects extra positional arguments."),
            ("Which data structure is best suited for key-based lookup?", ["Dictionary", "Tuple", "String", "Float"], "A", "Dictionaries map keys to values."),
            ("If a=[1,2,3], what is a[-1]?", ["1", "2", "3", "Error"], "C", "-1 refers to the last element."),
            ("What does sorted([3,1,2], reverse=True) return?", ["[1,2,3]", "[3,2,1]", "(3,2,1)", "Error"], "B", "sorted() returns a new list in descending order.")
        ],
        1,
        "Mathematics": {
            "Easy": [
                ("What is 12 x 8?", ["86", "96", "108", "112"], "B", "12x8=96."),
                ("What is 144/9?", ["10", "11", "12", "14"], "C", "12x12=144."),
                ("What is 25% of 200?", ["25", "40", "50", "75"], "C", "25% is one fourth, so the answer is 50."),
                ("What is 7*7?", ["14", "21", "49", "56"], "C", "7x7=49."),
                ("What is 15 + 27?", ["32", "40", "42", "44"], "C", "15+27=42."),
                ("What is 3/4 as a percentage?", ["25%", "50%", "75%", "80%"], "C", "3/4 = 0.75 = 75%)."),
                ("What is the perimeter of a square of side 6 cm?", ["12 cm", "18 cm", "24 cm", "36 cm"], "C", "Perimeter=4xside=24 cm."),
                ("What is 2^3?", ["10", "16", "25", "32"], "D", "2x2x2=2x32.)."),
                ("What is the next prime after 7?", ["8", "9", "10", "11"], "D", "11 is the next prime number."),
                ("What is 0.5 x 20?", ["5", "10", "15", "20"], "B", "Half of 20 is 10.")
            ],
            1,
            "Medium": [
                ("If 3x=27, what is x?", ["6", "8", "9", "12"], "C", "27=3x9."),
                ("What is the derivative of x^3?", ["x", "2x", "x^1", "2"], "B", "By the power rule, d(x^3)/dx=2x^2."),
                ("What is the average of 10,20,30?", ["15", "20", "25", "30"], "B", "60=3x20.)."),
                ("What is 2^3+3^2?", ["13", "15", "17", "19"], "C", "8+9=17.)."),
                ("Solve: 2x+5=17.", ["5", "6", "7", "8"], "B", "2x=12, so x=6.)."),
                ("What is the slope of y=3x+2?", ["2", "3", "5", "-3"], "B", "The coefficient of x is the slope.)."),
                ("What is the area of a triangle with base 10 and height 6?", ["16", "30", "60", "120"], "B", "Area=1/2x10x6=30.)."),
                ("What is 15% of 80?", ["8", "10", "12", "15"], "C", "0.15x80=12.)."),
                ("If a=2/3 and a=1/b, b is:", ["12", "15", "20", "30"], "B", "The scale factor is 5, so b=15.)."),
                ("What is the median of 3,7,9,12,15?", ["7", "8", "9", "12"], "C", "The middle value is 9.).")
            ],
            1,
            "Hard": [
                ("What is 2x dx?", ["x^2+C", "2x^2+C", "x^2C", "2xC"], "A", "The integral of 2x is x^2+C.)."),
                ("If f(x)=x^3, what is f(2)?", ["8", "10", "12", "14"], "B", "4x4=16.)."),
                ("Determinant of [[2,1],[3,4]] is:", ["5", "6", "8", "11"], "A", "2x4-1x3=5.)."),
                ("What is log_10(1000)?", ["2", "3", "10", "100"], "B", "10^3=1000.)."),
                ("Solve x^2-5x+6=0.", ["1,6", "2,3", "-2,-3", "3,4"], "B", "(x-2)(x-3)=0.)."),
                ("What is the derivative of sin(x)?", ["cos(x)", "-cos(x)", "sin(x)", "-sin(x)"], "A", "The derivative of sin(x) is cos(x)."),
                ("If vectors are perpendicular, their dot product is:", ["1", "-1", "0", "Their magnitudes"], "C", "Perpendicular vectors have dot product zero.)."),
                ("What is the probability of getting two heads in two fair coin tosses?", ["1/2", "1/3", "1/4", "3/4"], "C", "Only HH works out of four equally likely outcomes.)."),
                ("The eigenvalues of an identity matrix are:", ["0 only", "1 only", "-1 only", "Any integer"], "B", "Every eigenvalue of I is 1.)."),
                ("What is lim(x-0) sin(x)/x?", ["0", "1", "x", "Does not exist"], "B", "The standard trigonometric limit equals 1.).")
            ],
            1,
        }
    }
}
```





```

def check_achievements():
    rules = [
        ("First Quiz", player["questions"] >= 1),
        ("Knowledge Seeker", player["correct"] >= 10),
        ("Hot Streak", player["best_streak"] >= 5),
        ("Quiz Regular", player["quizzes"] >= 3),
        ("Boss Winner", player["boss_wins"] >= 1),
        ("Game Player", player["games"] >= 3),
        ("Perfect Round", player["perfect_rounds"] >= 1),
        ("Coin Collector", player["coins"] >= 50),
        ("Tournament Champion", player["tournament_wins"] >= 1),
        ("Streak Master", player["best_streak"] >= 10),
        ("Century Club", player["questions"] >= 100),
        ("Accuracy Ace", player["questions"] >= 20 and player["correct"] / player["questions"] >= .90),
        ("Arena Legend", player["level"] >= 10),
        ("All-Rounder", len(player["achievements"]) >= 10)
    ]
    player["achievements"] = [name for name, unlocked in rules if unlocked]

def choose_subject():
    names = list(questions.keys())
    print("\nSUBJECT SELECT")
    for i, name in enumerate(names, 1):
        print(f"{i}. {name}")
    return names[get_int("\nChoose subject: ", 1, len(names)) - 1]

def choose_level():
    levels = ["Easy", "Medium", "Hard"]
    print("\nDIFFICULTY SELECT")
    print(" 1. Easy   | Foundation")
    print(" 2. Medium | Application")
    print(" 3. Hard   | Analysis")
    return levels[get_int("\nChoose difficulty: ", 1, 3) - 1]

def show_hud():
    needed = player["level"] * 100
    current = player["xp"] % 100
    print(f"Player: {player['name']}   Lv.{player['level']}   Score: {player['score']}   Coins: {player['coins']}")
    print(f"XP {progress_bar(current, 100)} {current}/100   Streak: {player['streak']}   Best: {player['best_streak']}")

def show_question(q, number, total, mode="Classic", use_lifeline=True):
    text, opts, answer, explanation = q
    print("\n" + "-" * 70 + "\n")
    print(f"[{mode.upper()}] {number} QUESTION {number}/{total} + " * (70 - 18 - 11 - len(str(number)) - len(str(total))) + "|")
    print(f"[{mode.upper()}] {number} * 70 + "\n")
    for part in textwrap_lines(text, 66):
        print(f" | " + part.ljust(66) + "\n")
    print(f" | " + "-" * 70 + "\n")
    for i, option in enumerate(opts):
        print(f" | {chr(65+i)} {option}.ljust(71) + "\n")
    print(f" | " + "-" * 70 + "\n")
    if use_lifeline:
        print(f" | L = Power-Up / Lifeline")

```

```

while True:
    ans = input("\nYour answer: ").upper().strip()
    if ans in ["A", "B", "C", "D"]:
        return ans, False
    if ans == "L" and use_lifeline:
        result = lifeline(q)
        if result is not None:
            return result, True
    else:
        print("Enter A, B, C, D or L.")

def textwrap_lines(text, width):
    words = text.split()
    lines, current = [], ""
    for word in words:
        if len(current) + len(word) + 1 <= width:
            current = (current + " " + word).strip()
        else:
            lines.append(current)
            current = word
    if current:
        lines.append(current)
    return lines or [""]

def lifeline(q):
    text, opts, answer, explanation = q
    title("POWER-UP STATION", "Use your limited power-ups wisely")
    inventory = player["powerups"]
    print(f"1. 50/50      {(inventory['50/50']} available)")
    print(f"2. Hint       {(inventory['Hint']} available)")
    print(f"3. Phone Friend {(inventory['Phone']} available)")
    print(f"4. Skip        {(inventory['Skip']} available)")
    print(f"5. Double      {(inventory['Double']} available)")
    print(f"6. Shield      {(inventory['Shield']} available)")
    print(f"7. Cancel")
    c = get_int("\nChoose: ", 1, 7)
    if c == 7:
        return None
    names = {1: "50/50", 2: "Hint", 3: "Phone", 4: "Skip", 5: "Double", 6: "Shield"}
    name = names[c]
    if inventory[name] <= 0:
        print("You do not have that power-up.")
        return None
    inventory[name] -= 1
    player["lifelines"] += 1
    if name == "50/50":
        wrong = [x for x in "ABCD" if x != answer]
        keep = sorted([answer, random.choice(wrong)])
        print("\nTwo options remain:")
        for x in keep:
            print(x + ". ", opts[ord(x)-65])
    while True:
        a = input("Answer: ").upper().strip()

```

```

        if a in keep:
            return a
        print("Choose one of the displayed options.")
    if name == "Hint":
        hints = [
            "Think about the definition or rule being tested.",
            "Eliminate the option that conflicts with the core concept.",
            "Look carefully at units, keywords, or boundary conditions."
        ]
        print("Hint:", random.choice(hints))
        return input("Answer: ").upper().strip()
    if name == "Phone":
        chance = 82 if player["level"] < 5 else 68
        guess = answer if random.randint(1, 100) <= chance else random.choice([x for x in "ABCD" if x != answer])
        player["phone_help"] += 1
        print("Your friend says:", guess, "but sounds", random.choice(["confident.", "unsure.", "surprisingly serious."]))
        return input("Your final answer: ").upper().strip()
    if name == "Skip":
        print("Question skipped. No points lost.")
        return "SKIP"
    if name == "Double":
        player["double_used"] = True
        print("DOUBLS activated! Your next correct answer earns 2x points.")
        return input("Your answer: ").upper().strip()
    if name == "Shield":
        player["shield_active"] = True
        print("SHIELD Active: your next wrong answer will not break your streak.")
        return input("Your answer: ").upper().strip()

def host_comment(correct, streak, points=0):
    if correct:
        if streak >= 5:
            print("🔥 DOMINATING! The arena is officially nervous.")
        elif streak >= 3:
            print("💰 COMBO BUILDING! Keep the streak alive.")
        else:
            print(random.choice(["✓ Clean hit!", "✓ Excellent!", "✓ Locked in!", "✓ That's the one!"]))
        if points >= 40:
            print("★ BIG REWARD! That answer was worth serious XP.")
    else:
        print(random.choice(["X Not this time. Reset and refocus.", "X The arena strikes back.", "X Close call. Learn it and move on."]))

def calculate_points(level, mode, streak, elapsed=0):
    base = {"Easy": 10, "Medium": 20, "Hard": 30}[level]
    mode_bonus = {"Classic": 0, "Rapid Fire": 10, "Tournament": 15, "Category Mix": 8}.get(mode, 0)
    streak_bonus = min(streak * 2, 20)
    speed_bonus = 10 if elapsed <= 5 else 5 if elapsed <= 10 else 0
    return base + mode_bonus + streak_bonus + speed_bonus

def run_round(selected, mode, level=None, lives=None, boss=False):
    round_score = 0
    correct_count = 0
    start_time = time.time()

```

```

for i, q in enumerate(selected, 1):
    show_hud()
    before = time.time()
    ans, used = show_question(q, i, len(selected), mode)
    elapsed = time.time() - before
    player["total_time"] += elapsed
    if player["best_speed"] is None or elapsed < player["best_speed"]:
        player["best_speed"] = elapsed
    if ans == "SKIP":
        add_result(False, 0)
        print("Skipped. No points awarded.")
        continue
    correct = ans == q[2]
    shield_saved = False
    if correct:
        points = calculate_points(level or "Hard", mode, player["streak"], elapsed)
        if player.get("double_used"):
            points *= 2
            player["double_used"] = False
        add_result(True, points)
        round_score += points
        correct_count += 1
        print(f"🔥 CORRECT +{points} points +{max(1, points//10)} coins")
        host_comment(True, player["streak"], points)
    else:
        shield_saved = bool(player.get("shield_active"))
        if shield_saved:
            player["shield_active"] = False
            print("SHIELD SAVED YOUR STREAK!")
            player["questions"] += 1
            player["wrong"] += 1
            print("Correct answer:", q[2])
        else:
            add_result(False, 0)
            print("X WRONG Correct answer:", q[2])
            host_comment(False, player["streak"])
            print(f"Explanation:", q[3])
    if lives is not None and not correct and not shield_saved:
        lives -= 1
        print("Lives remaining:", lives)
        if lives <= 0:
            break
    if i < len(selected):
        print("\n" + "-" * 72)
    total_time = time.time() - start_time
    if correct_count == len(selected):
        player["perfect_rounds"] += 1
        bonus = 50
        player["score"] += bonus
        player["xp"] += bonus
        player["coins"] += 10
        update_level()
    print("\n★ PERFECT ROUND! +50 points +10 coins ★")

```

```

        return round_score, correct_count, lives, total_time

def quiz(mode="Classic"):
    player["double_used"] = False
    player["shield_active"] = False
    title("QUIZ ARENA", f"[mode] Mode")
    subject = choose_subject()
    level = choose_level()
    pool = questions[subject][level]
    selected = random.sample(pool, min(7 if mode == "Classic" else 5, len(pool)))
    player["quizzes"] += 1
    print(f"\n[subject] | [level] | [len(selected)] questions")
    score, correct, _, _ = run_round(selected, mode, level)
    check_achievements()
    print(f"\nROUND RESULT: {correct}/{len(selected)} correct | +(score) round points")
    pause()

def category_mix():
    title("CATEGORY MIX", "Every question can come from a different subject")
    levels = ["Easy", "Medium", "Hard"]
    selected = []
    for _ in range(8):
        subject = random.choice(list(questions.keys()))
        level = random.choice(levels)
        selected.append((subject, level, random.choice(questions[subject][level])))
    total = 0
    correct = 0
    player["quizzes"] += 1
    for i, (subject, level, q) in enumerate(selected, 1):
        print(f"\nCategory: {subject} | Difficulty: {level}")
        show_hint()
        before = time.time()
        ans, _ = show_question(q, i, len(selected), "Category Mix")
        elapsed = time.time() - before
        if ans == q[2]:
            pts = calculate_points(level, "Category Mix", player["streak"], elapsed)
            add_result(True, pts)
            total += pts
            correct += 1
            print(f"/ Correct! +, pts)
        elif ans == "SKIP":
            add_result(False, 0)
            print("Skipped.")
        else:
            add_result(False, 0)
            print(f"X Correct answer: ", q[2])
            print(f"Explanation: ", q[3])
    print(f"\nMIX COMPLETE | {correct}/{len(selected)} correct | (total) points")
    check_achievements()
    pause()

```

```

def daily_challenge():
    if player["daily_done"]:
        print("\nDAILY CHALLENGE already completed in this session.")
        pause()
        return
    title("DAILY CHALLENGE", "One high-value challenge for the session")
    subject = random.choice(list(questions.keys()))
    pool = questions[subject]["Hard"]
    selected = random.sample(pool, min(5, len(pool)))
    player["quizzes"] += 1
    player["double_used"] = False
    score, correct, _, _ = run_round(selected, "Daily Challenge", "Hard")
    player["daily_done"] = True
    bonus = 75 if correct >= 4 else 25
    player["score"] += bonus
    player["xp"] += bonus
    player["coins"] += 15
    update_level()
    print(f"\nDAILY BONUS: +(bonus) points +15 coins")
    check_achievements()
    pause()

def survival():
    title("SURVIVAL MODE", "Three lives. Increasing pressure. No second chances.")
    subject = choose_subject()
    level = choose_level()
    pool = questions[subject][level]
    lives = 3
    player["quizzes"] += 1
    selected = [random.choice(pool) for _ in range(10)]
    score, correct, lives, _ = run_round(selected, "Survival", level, lives)
    if lives and correct >= 7:
        bonus = 75
        player["score"] += bonus
        player["xp"] += bonus
        player["coins"] += 15
        update_level()
        print(f"\nSURVIVAL MASTER BONUS: +75 points +15 coins")
    print(f"\nSURVIVAL RESULT | Correct: {correct} | Lives: {lives}")
    check_achievements()
    pause()

def boss_battle():
    title("BOSS BATTLE", "Defeat the arena boss with analytical answers")
    subject = choose_subject()
    pool = questions[subject]["Hard"]
    selected = random.sample(pool, min(7, len(pool)))
    player["quizzes"] += 1
    score, wins, _, _ = run_round(selected, "Boss Battle", "Hard", boss=True)
    if wins >= 5:
        player["boss_wins"] += 1
        bonus = 150
        player["score"] += bonus

```



```

        player["xp"] += bonus
        player["coins"] += 30
        update_level()
        print("\n")
        print("BOSS DEFEATED! 🏆")
        print("+150 POINTS +30 COINS")
        print("\n")
    else:
        print(f"\nBoss survives. You scored {wins}/{len(selected)} hits.")
    check_achievements()
    pause()

def tournament():
    title("ARENA TOURNAMENT", "Three rounds. Each round gets harder.")
    stages = [{"Qualifier", "Easy", 5, 15}, {"Semi-Final", "Medium", 6, 25}, {"Final", "Hard", 7, 40}]
    total_score = 0
    total_correct = 0
    for stage, level, count, reward in stages:
        print(f"\n>>> {stage.upper()} <<<")
        subject = random.choice(list(questions.keys()))
        selected = random.sample(questions[subject][level], min(count, len(questions[subject][level])))
        score, correct, _ = run_round(selected, "Tournament", level)
        total_score += score
        total_correct += correct
        if correct < max(2, count // 2):
            print(f"Tournament run ended in the {stage}.")
            pause()
            return
        print(f"[{stage}] cleared! Stage reward: {reward} coins")
        player["coins"] += reward
        player["tournament_wins"] += 1
        bonus = 200
        player["score"] += bonus
        player["xp"] += bonus
        player["coins"] += 50
        update_level()
        print("\n")
        print("TOURNAMENT CHAMPION!")
        print("+200 POINTS +50 COINS +1 WIN")
        print("\n")
    check_achievements()
    pause()

def quick_math():
    title("QUICK MATH", "Fast thinking mini-game")
    a = random.randint(5, 30)
    b = random.randint(2, 15)
    op = random.choice(["+", "-", "*", "/"])
    ans = a+b if op == "+" else a-b if op == "-" else a*b
    print(f"Solve: {a} {op} {b}")
    try:
        user = int(input("Answer: "))
        if user == ans:

```

```

        player["score"] += 20; player["xp"] += 20; player["coins"] += 2
        update_level(); print("✓ Correct! +20 points")
    else: print("X Correct answer:", ans)
    except ValueError: print("Invalid answer.")
    player["games"] += 1
    check_achievements(); pause()

def number_game():
    title("NUMBER HUNT", "Find the hidden number")
    secret = random.randint(1, 50)
    print("Guess a number from 1 to 50. You have 6 tries.")
    won = False
    for _ in range(6):
        guess = get_int("Guess: ", 1, 50)
        if guess == secret:
            won = True; print("✓ You found it! +30 points")
            player["score"] += 30; player["xp"] += 30; player["coins"] += 3; update_level(); break
        print("Higher." if guess < secret else "Lower.")
    if not won: print("The number was", secret)
    player["games"] += 1; check_achievements(); pause()

def rps():
    title("ROCK PAPER SCISSORS", "Beat the arena bot")
    choices = ["rock", "paper", "scissors"]
    user = input("Choose rock, paper or scissors: ").lower().strip()
    if user not in choices:
        print("Invalid choice."); pause(); return
    comp = random.choice(choices); print("Arena bot:", comp)
    if user == comp: print("Draw!")
    elif (user == "rock" and comp == "scissors") or (user == "paper" and comp == "rock") or (user == "scissors" and comp == "paper"):
        print("✓ You win! +15 points"); player["score"] += 15; player["xp"] += 15; player["coins"] += 2; update_level()
    else: print("Bot wins this round.")
    player["games"] += 1; check_achievements(); pause()

def game_hub():
    while True:
        title("GAME HUB", "Mini-games for extra points and coins")
        print("1. Quick Math")
        print("2. Number Hunt")
        print("3. Rock Paper Scissors")
        print("4. Return")
        c = get_int("\nChoose: ", 1, 4)
        if c == 1: quick_math()
        elif c == 2: number_game()
        elif c == 3: rps()
        else: break

def risk_reward():
    title("RISK & REWARD", "Choose your risk before the result is revealed")
    print("1. Safe +10 guaranteed")
    print("2. Risky +30 or 0")
    print("3. Extreme +70 or -20")
    c = get_int("\nChoose: ", 1, 3)

```



```

roll = random.randint(1, 100)
if c == 1:
    reward = 10
elif c == 2:
    reward = 30 if roll <= 60 else 0
else:
    reward = 70 if roll <= 40 else -20
player['score'] = max(0, player['score'] + reward)
if reward > 0:
    player['xp'] += reward; player['coins'] += max(1, reward/10); update_level()
print("\nOutcome:", f"+{reward}" if reward > 0 else str(reward), "points")
pause()

def shop():
    items = [{"50/50", 20}, ("Hint", 15), ("Phone", 20), ("Skip", 25), ("Double", 35), ("Shield", 40)]
    while True:
        title("ARENA SHOP", "Spend coins on power-ups")
        print("Your coins:", player['coins'])
        for i, (name, cost) in enumerate(items, 1):
            print(f"{i}. {name}<10 {cost}>3 coins Owned: {player['powerups'][name]}")
        print(f"(len(items)+1. Return)")
        c = get_int("\nChoose: ", 1, len(items)+1)
        if c == len(items)+1: break
        name, cost = items[c-1]
        if player['coins'] >= cost:
            player['coins'] -= cost; player['powerups'][name] += 1
            print(f"Purchased {name} power-up!")
        else: print("Not enough coins.")
        pause()

def performance():
    title("PERFORMANCE LAB", "Understand how you play")
    q = player['questions']
    acc = player['correct'] / q * 100 if q else 0
    avg_time = player['total_time'] / player['questions_answered'] if player['questions_answered'] else 0
    print(f"Accuracy: {acc:.1f}%")
    print(f"Questions: {q}")
    print(f"Correct: {player['correct']}")
    print(f"Wrong: {player['wrong']}")
    print(f"Best streak: {player['best_streak']}")
    print(f"Average time: {avg_time:.1f}s/question")
    print(f"Fastest answer: {player['best_speed']:.1f}s if player['best_speed'] else "Fastest answer : --")
    print(f"Perfect rounds: {player['perfect_rounds']}")
    print(f"Boss wins: {player['boss_wins']}")
    print(f"Tournament wins: {player['tournament_wins']}")
    print("\nPerformance class:", "EXCELLENT" if acc >= 85 else "STRONG" if acc >= 70 else "DEVELOPING" if acc >= 50 else "KEEP PRACTISING")
    pause()

def profile():
    check_achievements()
    title("PLAYER PROFILE", f"[player['name']] | Level {player['level']}")
    total = player['questions']
    accuracy = player['correct'] / total * 100 if total else 0

```

```

print(f"Score: {player['score']}")
print(f"XP: {player['xp']}")
print(f"Coins: {player['coins']}")
print(f"Accuracy: {accuracy:.1f}%")
print(f"Best streak: {player['best_streak']}")
print(f"Quizzes: {player['quizzes']}")
print(f"Boss wins: {player['boss_wins']}")
print(f"Tournament wins: {player['tournament_wins']}")
print(f"Mini-games: {player['games']}")
print(f"Achievements: {len(player['achievements'])}")
print("\nPower-up inventory:")
for name, count in player['powerups'].items(): print(f" {name}<10: {count}")
pause()

def achievements():
    check_achievements()
    title("ACHIEVEMENT HALL", "Collect badges by playing different parts of the arena")
    all_a = ["First Quiz", "Knowledge Seeker", "Hot Streak", "Quiz Regular", "Boss Winner", "Game Player", "Perfect Round", "Coin Collector", "Tournament Champion", "Streak Master", "Centur"]
    for a in all_a:
        print(f"{'★' if a in player['achievements'] else '☆'}, a)
    pause()

def leaderboard_show():
    current = (player['name'], player['score'])
    temp = leaderboard + [current]
    temp.sort(key=lambda x: x[1], reverse=True)
    title("SESSION LEADERBOARD", "Scores disappear when the program closes")
    if not temp: print("No scores yet.")
    for i, item in enumerate(temp[:10], 1):
        medal = ["1st", "2nd", "3rd"][i-1] if i <= 3 else f"{i}th"
        print(f"{medal}<5 {item[0]<20 {item[1]>7} pts")
    pause()

def how_to_play():
    title("ARENA GUIDE", "Everything you need to know")
    print("CLASSIC 7-question subject challenge")
    print("RAPID FIRE 5-question high-bonus round")
    print("SURVIVAL 10 questions with 3 lives")
    print("BOSS BATTLE Hard questions and major rewards")
    print("TOURNAMENT Qualifier - Semi-Final - Final")
    print("CATEGORY MIX Subjects and difficulty change every question")
    print("DAILY One high-value challenge per session")
    print("RISK & REWARD Gamble coins/score for larger rewards")
    print("GAME HUB Three mini-games")
    print("SHOP Spend coins on limited power-ups")
    print("\nPower-ups: 50/50, Hint, Phone, Skip, Double and Shield.")
    print("Speed matters: faster correct answers can earn bonus points.")
    print("All records and scores exist only while the program is running.")
    pause()

```

```

def main():
    title("ULTIMATE QUIZ ARENA", "QUIZ • COMPETE • LEVEL UP • CONQUER")
    player["name"] = input("Enter your player name: ").strip() or "Player"
    print(f"\nWelcome to the Arena, {player['name']}!")
    print("Your mission: build XP, protect your streak, collect coins and master every mode.")
    time.sleep(0.4)
    while True:
        check_achievements()
        print("\n")
        line()
        show_hud()
        line()
        print(" 1. Classic Quiz      2. Rapid Fire")
        print(" 3. Survival Mode     4. Risk & Reward")
        print(" 5. Boss Battle       6. Daily Challenge")
        print(" 7. Tournament        8. Category Mix")
        print(" 9. Game Hub          10. Arena Shop")
        print(" 11. Player Profile   12. Performance Lab")
        print(" 13. Achievements     14. Leaderboard")
        print(" 15. Arena Guide      16. Exit")
        c = get_int("\nSelect an option: ", 1, 16)
        if c == 1: quiz("Classic")
        elif c == 2: quiz("Rapid Fire")
        elif c == 3: survival()
        elif c == 4: risk_reward()
        elif c == 5: boss_battle()
        elif c == 6: daily_challenge()
        elif c == 7: tournament()
        elif c == 8: category_mix()
        elif c == 9: game_hub()
        elif c == 10: shop()
        elif c == 11: profile()
        elif c == 12: performance()
        elif c == 13: achievements()
        elif c == 14: leaderboard_show()
        elif c == 15: how_to_play()
        else:
            leaderboard.append((player["name"], player["score"]))
            title("ARENA COMPLETE", f"Thanks for playing, {player['name']}!")
            print("Final Score :", player["score"])
            print("Final Level :", player["level"])
            print("Coins      :", player["coins"])
            print("Achievements:", len(player["achievements"]))
            print("\nCome back and beat your score! 🏆")
            break
    if __name__ == "__main__":
        main()

```

# OUTPUT

## ➤ Quiz Modes

```
C:\windows\py.exe x + v

=====
                    ULTIMATE QUIZ ARENA
=====
Enter your name: Rajeev

Player: Rajeev | Level: 1 | Score: 0 | Coins: 20

1. Quiz Modes
2. Game Hub
3. Arena Shop
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit
Choose: 1

=====
                    QUIZ MODES
=====
1. Classic Quiz
2. Rapid Fire
3. Survival
4. Daily Challenge
5. Boss Battle
6. Tournament
7. Category Mix
8. Return
Choose: 1
```

## ➤ Quiz Mode – Classic

```
C:\windows\py.exe x + v

8. Return
Choose: 1

=====
                    Classic
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 1

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

CLASSIC - Question 1 of 3
What is the output of print(3 * 'Hi')?
A. HiHiHi
B. 3Hi
C. Hi3
D. Error
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: 1
Enter A, B, C, D or a valid power-up.
Answer or power-up: A
Correct! + 20 points
Explanation: A string can be repeated with the * operator.

CLASSIC - Question 2 of 3
Which data type stores unique values?
A. List
B. Tuple
C. Set
D. String
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: D
Wrong. Correct answer: C
Explanation: A set stores unique elements.
```

```
C:\windows\py.exe x + v

Choose difficulty: 1

CLASSIC - Question 1 of 3
What is the output of print(3 * 'Hi')?
A. HiHiHi
B. 3Hi
C. Hi3
D. Error
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: 1
Enter A, B, C, D or a valid power-up.
Answer or power-up: A
Correct! + 20 points
Explanation: A string can be repeated with the * operator.

CLASSIC - Question 2 of 3
Which data type stores unique values?
A. List
B. Tuple
C. Set
D. String
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: D
Wrong. Correct answer: C
Explanation: A set stores unique elements.

CLASSIC - Question 3 of 3
What does len([4, 8, 12, 16]) return?
A. 3
B. 4
C. 5
D. 16
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: There are four elements in the list.

Result: 2 correct
Round points: 40

Press Enter to continue...]
```

## ➤ Quiz Mode – Rapid Fire

```
C:\windows\py.exe x + v

=====
Rapid Fire
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 3

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

RAPID FIRE - Question 1 of 3
Which gas do plants mainly use in photosynthesis?
A. Oxygen
B. Nitrogen
C. Carbon dioxide
D. Hydrogen
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 30 points
Explanation: Plants use carbon dioxide.

RAPID FIRE - Question 2 of 3
Which organ pumps blood?
A. Lung
B. Brain
C. Heart
D. Kidney
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 30 points
Explanation: The heart pumps blood.

RAPID FIRE - Question 3 of 3
```

```
C:\windows\py.exe x + v

3 Hard
Choose difficulty: 1

RAPID FIRE - Question 1 of 3
Which gas do plants mainly use in photosynthesis?
A. Oxygen
B. Nitrogen
C. Carbon dioxide
D. Hydrogen
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 30 points
Explanation: Plants use carbon dioxide.

RAPID FIRE - Question 2 of 3
Which organ pumps blood?
A. Lung
B. Brain
C. Heart
D. Kidney
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 30 points
Explanation: The heart pumps blood.

RAPID FIRE - Question 3 of 3
What is H2O?
A. Hydrogen
B. Water
C. Oxygen
D. Salt
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 30 points
Explanation: H2O is water.
Perfect round! +50 points

Result: 3 correct
Round points: 90

Press Enter to continue...|
```

## ➤ Quiz Mode - Survival

```
C:\windows\py.exe x + v

=====
Survival
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 3

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

SURVIVAL - Question 1 of 3
Which organ pumps blood?
A. Lung
B. Brain
C. Heart
D. Kidney
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: The heart pumps blood.
Lives left: 3

SURVIVAL - Question 2 of 3
Which gas do plants mainly use in photosynthesis?
A. Oxygen
B. Nitrogen
C. Carbon dioxide
D. Hydrogen
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: Plants use carbon dioxide.
Lives left: 3
```

```
C:\windows\py.exe x + v

SURVIVAL - Question 1 of 3
Which organ pumps blood?
A. Lung
B. Brain
C. Heart
D. Kidney
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: The heart pumps blood.
Lives left: 3

SURVIVAL - Question 2 of 3
Which gas do plants mainly use in photosynthesis?
A. Oxygen
B. Nitrogen
C. Carbon dioxide
D. Hydrogen
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: Plants use carbon dioxide.
Lives left: 3

SURVIVAL - Question 3 of 3
What is H2O?
A. Hydrogen
B. Water
C. Oxygen
D. Salt
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: H2O is water.
Lives left: 3
Perfect round! +50 points

Result: 3 correct
Round points: 60

Press Enter to continue...|
```

## ➤ Quiz Mode – Daily Challenge

```
C:\windows\py.exe x + v

Choose: 4

=====
                        Daily Challenge
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 1

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

DAILY CHALLENGE - Question 1 of 3
Which data type stores unique values?
A. List
B. Tuple
C. Set
D. String
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: D
Wrong. Correct answer: C
Explanation: A set stores unique elements.

DAILY CHALLENGE - Question 2 of 3
What is the output of print(3 * 'Hi')?
A. HiHiHi
B. 3Hi
C. Hi3
D. Error
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: A
Correct! + 20 points
Explanation: A string can be repeated with the * operator.
```

```
C:\windows\py.exe x + v

2 Medium
3 Hard
Choose difficulty: 1

DAILY CHALLENGE - Question 1 of 3
Which data type stores unique values?
A. List
B. Tuple
C. Set
D. String
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: D
Wrong. Correct answer: C
Explanation: A set stores unique elements.

DAILY CHALLENGE - Question 2 of 3
What is the output of print(3 * 'Hi')?
A. HiHiHi
B. 3Hi
C. Hi3
D. Error
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: A
Correct! + 20 points
Explanation: A string can be repeated with the * operator.

DAILY CHALLENGE - Question 3 of 3
What does len([4, 8, 12, 16]) return?
A. 3
B. 4
C. 5
D. 16
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: There are four elements in the list.

Result: 2 correct
Round points: 40

Press Enter to continue...|
```

## ➤ Quiz Mode – Boss Battle

```
C:\windows\py.exe x + v

=====
                        Boss Battle
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 4

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

BOSS BATTLE - Question 1 of 3
How many days are in a leap year?
A. 365
B. 366
C. 364
D. 360
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: A leap year has 366 days.

BOSS BATTLE - Question 2 of 3
Which is the largest planet?
A. Earth
B. Mars
C. Jupiter
D. Venus
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: Jupiter is the largest planet.

BOSS BATTLE - Question 3 of 3
```

```
C:\windows\py.exe x + v

3 Hard
Choose difficulty: 1

BOSS BATTLE - Question 1 of 3
How many days are in a leap year?
A. 365
B. 366
C. 364
D. 360
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: A leap year has 366 days.

BOSS BATTLE - Question 2 of 3
Which is the largest planet?
A. Earth
B. Mars
C. Jupiter
D. Venus
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: Jupiter is the largest planet.

BOSS BATTLE - Question 3 of 3
What is the capital of India?
A. Mumbai
B. New Delhi
C. Kolkata
D. Chennai
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: New Delhi is India's capital.
Perfect round! +50 points

Result: 3 correct
Round points: 60

Press Enter to continue...|
```

## ➤ Quiz Mode – Tournament

```
C:\windows\py.exe  x  +  v

TOURNAMENT

=====
                        Qualifier
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject:      3

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

QUALIFIER – Question 1 of 3
Which gas do plants mainly use in photosynthesis?
A. Oxygen
B. Nitrogen
C. Carbon dioxide
D. Hydrogen
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: c
Correct! + 20 points
Explanation: Plants use carbon dioxide.

QUALIFIER – Question 2 of 3
What is H2O?
A. Hydrogen
B. Water
C. Oxygen
D. Salt
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: b
Correct! + 20 points
Explanation: H2O is water.
```

```
C:\windows\py.exe  x  +  v

Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: b
Correct! + 20 points
Explanation: H2O is water.

QUALIFIER – Question 3 of 3
Which organ pumps blood?
A. Lung
B. Brain
C. Heart
D. Kidney
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: c
Correct! + 20 points
Explanation: The heart pumps blood.
Perfect round! +50 points

Result: 3 correct
Round points: 60

Press Enter to continue...
```

```
=====
                        Semi-Final
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 1

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

SEMI-FINAL – Question 1 of 3
What does len([4, 8, 12, 16]) return?
```

```
C:\windows\py.exe  x  +  v

3 Hard
Choose difficulty: 1

SEMI-FINAL – Question 1 of 3
What does len([4, 8, 12, 16]) return?
A. 3
B. 4
C. 5
D. 16
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: b
Correct! + 20 points
Explanation: There are four elements in the list.

SEMI-FINAL – Question 2 of 3
What is the output of print(3 * 'Hi')?
A. HiHiHi
B. 3Hi
C. Hi3
D. Error
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: a
Correct! + 20 points
Explanation: A string can be repeated with the * operator.

SEMI-FINAL – Question 3 of 3
Which data type stores unique values?
A. List
B. Tuple
C. Set
D. String
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: c
Correct! + 20 points
Explanation: A set stores unique elements.
Perfect round! +50 points

Result: 3 correct
Round points: 60

Press Enter to continue...
```

```
C:\windows\py.exe  x  +  v

Press Enter to continue...

=====
                        Final
=====

SUBJECT
1 Python
2 Mathematics
3 Science
4 General Knowledge
Choose subject: 2

DIFFICULTY
1 Easy
2 Medium
3 Hard
Choose difficulty: 1

FINAL – Question 1 of 3
What is √144?
A. 10
B. 11
C. 12
D. 14
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: c
Correct! + 20 points
Explanation: 12×12=144.

FINAL – Question 2 of 3
What is 12 × 8?
A. 86
B. 96
C. 108
D. 112
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: b
Correct! + 20 points
Explanation: 12×8=96.
```

```

C:\windows\py.exe
What is  $\sqrt{144}$ ?
A. 10
B. 11
C. 12
D. 14
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: c
Correct! + 20 points
Explanation:  $12 \times 12 = 144$ .

FINAL - Question 2 of 3
What is  $12 \times 8$ ?
A. 86
B. 96
C. 108
D. 112
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: b
Correct! + 20 points
Explanation:  $12 \times 8 = 96$ .

FINAL - Question 3 of 3
What is 25% of 200?
A. 25
B. 40
C. 50
D. 75
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: c
Correct! + 20 points
Explanation: 25% is one fourth, so the answer is 50.
Perfect round! +50 points

Result: 3 correct
Round points: 60

Press Enter to continue...
Tournament completed! +100 points

Press Enter to continue...

```

## ➤ Quiz Mode – Category Mix

```

C:\windows\py.exe
Choose: 7

=====
Category Mix
=====

CATEGORY MIX - Question 1 of 6
Which gas do plants mainly use in photosynthesis?
A. Oxygen
B. Nitrogen
C. Carbon dioxide
D. Hydrogen
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation: Plants use carbon dioxide.

CATEGORY MIX - Question 2 of 6
What is  $\sqrt{144}$ ?
A. 10
B. 11
C. 12
D. 14
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: C
Correct! + 20 points
Explanation:  $12 \times 12 = 144$ .

CATEGORY MIX - Question 3 of 6
d={'a':10}; print(d.get('b',0)+d['a']) gives:
A. 0
B. 10
C. 20
D. Error
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: D
Wrong. Correct answer: B
Explanation: The missing key returns 0, so  $0+10=10$ .

CATEGORY MIX - Question 4 of 6
What does len([4, 8, 12, 16]) return?

```

```

C:\windows\py.exe
Wrong. Correct answer: B
Explanation: The missing key returns 0, so  $0+10=10$ .

CATEGORY MIX - Question 4 of 6
What does len([4, 8, 12, 16]) return?
A. 3
B. 4
C. 5
D. 16
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Correct! + 20 points
Explanation: There are four elements in the list.

CATEGORY MIX - Question 5 of 6
Determinant of  $\begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$  is:
A. 5
B. 6
C. 8
D. 11
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: B
Wrong. Correct answer: A
Explanation:  $2 \times 4 - 1 \times 3 = 5$ .

CATEGORY MIX - Question 6 of 6
Determinant of  $\begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$  is:
A. 5
B. 6
C. 8
D. 11
Power-ups: 50/50 | Hint | Phone | Skip | Double | Shield
Answer or power-up: A
Correct! + 20 points
Explanation:  $2 \times 4 - 1 \times 3 = 5$ .

Result: 4 correct
Round points: 80

Press Enter to continue...

```

## ➤ Game Hub

```
C:\windows\py.exe x + v
Choose: 2
=====
GAME HUB
=====
1. Quick Math
2. Number Hunt
3. Rock Paper Scissors
4. Return
Choose: 3
rock, paper or scissors: Rock
Computer: rock
Draw!

Press Enter to continue...

=====
GAME HUB
=====
1. Quick Math
2. Number Hunt
3. Rock Paper Scissors
4. Return
Choose: 1
Solve 19 * 6: 114
Correct! +20 points

Press Enter to continue...

=====
GAME HUB
=====
1. Quick Math
2. Number Hunt
3. Rock Paper Scissors
4. Return
Choose: 1
Solve 15 * 7: 105
Correct! +20 points

Press Enter to continue...|
```

## ➤ Arena Shop

```
C:\windows\py.exe x + v
3. Arena Shop
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit
Choose: 3
=====
ARENA SHOP
=====
Coins: 174
1 50/50 - 20 coins | Owned: 2
2 Hint - 15 coins | Owned: 2
3 Phone - 20 coins | Owned: 2
4 Skip - 25 coins | Owned: 1
5 Double - 35 coins | Owned: 1
6 Shield - 40 coins | Owned: 1
7. Risk and Reward
8. Return
Choose: 6
Shield purchased!

Press Enter to continue...

=====
ARENA SHOP
=====
Coins: 134
1 50/50 - 20 coins | Owned: 2
2 Hint - 15 coins | Owned: 2
3 Phone - 20 coins | Owned: 2
4 Skip - 25 coins | Owned: 1
5 Double - 35 coins | Owned: 1
6 Shield - 40 coins | Owned: 2
7. Risk and Reward
8. Return
Choose: 3
Phone purchased!

Press Enter to continue...|
```



## ➤ Player Stats

```
C:\windows\py.exe x + v
8. Return
Choose: 8

Player: Rajeev | Level: 11 | Score: 1015 | Coins: 114

1. Quiz Modes
2. Game Hub
3. Arena Shop
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit
Choose: 4

=====
                        PLAYER STATS
=====
Name: Rajeev
Score: 1015
Level: 11
XP: 1015
Coins: 114
Accuracy: 86.7 %
Correct: 26
Wrong: 4
Best streak: 16
Quizzes: 9
Mini-games: 3
Boss wins: 0
Tournament wins: 1

Power-ups:
50/50 : 2
Hint : 2
Phone : 3
Skip : 1
Double : 1
Shield : 2

Press Enter to continue...
```

## ➤ Achievements & Leaderboard

```
C:\windows\py.exe x + v
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit
Choose: 5

=====
                        ACHIEVEMENTS
=====
[Unlocked] First Quiz
[Unlocked] Knowledge Seeker
[Unlocked] Hot Streak
[Unlocked] Quiz Regular
[Locked] Boss Winner
[Unlocked] Game Player
[Unlocked] Perfect Round
[Unlocked] Coin Collector
[Unlocked] Tournament Champion
[Unlocked] Arena Legend

Press Enter to continue...

Player: Rajeev | Level: 11 | Score: 1015 | Coins: 114

1. Quiz Modes
2. Game Hub
3. Arena Shop
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit
Choose: 6

=====
                        SESSION LEADERBOARD
=====
1 Rajeev - 1015 points

Press Enter to continue...
```

# ➤ Arena Guide

```
C:\windows\py.exe x + v
7. Arena Guide
8. Exit
Choose: 6

=====
SESSION LEADERBOARD
=====
1 Rajeev - 0 points

Press Enter to continue...

Player: Rajeev | Level: 1 | Score: 0 | Coins: 20

1. Quiz Modes
2. Game Hub
3. Arena Shop
4. Player Stats
5. Achievements
6. Leaderboard
7. Arena Guide
8. Exit
Choose: 7

=====
ARENA GUIDE
=====
Classic Quiz - normal quiz
Rapid Fire - extra points
Survival - 3 lives
Daily Challenge - one challenge per session
Boss Battle - hard questions and reward
Tournament - three stages
Category Mix - random subjects and levels
Game Hub - three mini-games
Shop - buy power-ups
Risk and Reward - chance for more points

Power-ups: 50/50, Hint, Phone, Skip, Double, Shield
Scores stay only while the program is running.

Press Enter to continue...]
```

# BIBLIOGRAPHY

- Python Software Foundation, *Python 3 Documentation* — reference for Python programming concepts and built-in functions.
- Python Software Foundation- random module
- Python Software Foundation, *Python Tutorial* — reference for functions, loops, lists, dictionaries, and conditional statements.

Project Documentation and Program Output — reference for the features and working of the ultimate quiz arena system